

Implementation Of Convolutional Neural Network (CNN) in The Android-Based Application for Detecting Coffee Bean Maturity

Eka Imamul Muhlisin
Department of Electrical Engineering
UIN Sunan Gunung Djati
Bandung, Indonesia
1197070042@student.uinsgd.ac.id

Lia Kamelia
Department of Electrical Engineering
UIN Sunan Gunung Djati
Bandung, Indonesia
lia.kamelia@uinsgd.ac.id

Rin Rin Nirmalasari
Department of Electrical Engineering
UIN Sunan Gunung Djati
Bandung, Indonesia
rinrinm@uinsgd.ac.id

Ramdani Wahyu Sururie
Sharia and Law Faculty
UIN Sunan Gunung Djati
Bandung, Indonesia
ramdani.wahyu@uinsgd.ac.id

Abstract—The research aims to explore the implementation of artificial intelligence in developing an application that can accurately detect the maturity level of roasted coffee beans solely using a smartphone device. The research employs an AI-based approach by utilizing deep learning techniques to train a model on a dataset containing various roasted coffee bean maturity levels. The training data comprises visual images of coffee beans at different stages of maturity, enabling the model to discern patterns and characteristics associated with maturity levels. The application is built upon a classification system model for the maturity level of roasted coffee beans, employing an algorithm or convolutional neural network (CNN) method using the ResNet50 architecture. The utilization of the CNN method can enhance the accuracy of the model. This system is developed by employing digital image processing with data represented as coffee bean images. The smartphone application system uses Visual Studio Code, Flutter, and Android Studio software, incorporating image processing, colour feature extraction, and CNN classification methods. The model evaluation demonstrates an accuracy level of 100%, with precision, recall, and F1-Score values also reaching 100%. However, the accuracy of the ResNet50 model in classifying the maturity level of roasted coffee beans in real-time applications, based on the confusion matrix, is approximately 83.3%.

Keywords— coffee beans, artificial intelligence, smartphone, deep learning.

I. INTRODUCTION

The coffee shop business presents a promising opportunity for entrepreneurs in today's society. Indonesian communities, on average, exhibit a penchant for social gatherings and culinary delights. As the demand for coffee consumption continues to rise, establishing a coffee shop can be initiated either as a passion project or with a clear intention to generate potential income and other benefits.

Barista training plays a pivotal role in enhancing the competencies of coffee professionals to prepare and serve coffee proficiently following established standards. Through comprehensive training programs, baristas are equipped with the skills to manually brew coffee and craft espresso-based beverages, which customers commonly seek. Baristas are

tasked with discerning the type, variety, and degree of coffee bean maturity. It enables them to expertly prepare beverages manually or utilize espresso machines to serve espresso-based drinks.

To produce delicious coffee, coffee beans undergo a process known as coffee roasting. This crucial step significantly influences the quality of the coffee, contributing up to 30% to 40% of its taste and aroma. In Indonesia, coffee roasting typically involves three main maturity levels: light, medium, and dark. Conversely, on the international stage, there are eight more detailed levels, including light, cinnamon, medium, high, city, full city, French, and Italian, representing varying degrees of coffee roasting maturity.

With its architectural advantages, deep learning technology, a subset of machine learning, employs numerous layers during training, allowing for effective learning from vast amounts of data. Deep learning has emerged as a potent technique within artificial intelligence, particularly in object detection and semantic segmentation. Notably, there exist various deep learning algorithms, including Deep Neural Networks (DNN), Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN)[1].

This research selected the Convolutional Neural Network (CNN) algorithm for its efficacy in supervised deep-learning tasks, particularly with image data. CNNs are well-suited for detecting and recognizing objects in images, drawing inspiration from the visual processing mechanisms of humans and animals. Leveraging an architecture reminiscent of the visual cortex, CNNs have demonstrated remarkable results in image recognition[2].

Convolutional Neural Networks (CNNs) encompass various architectures, including LeNet 5, AlexNet, GoogLeNet (Inception), and ResNet, each with its unique advantages and disadvantages[3]. Therefore, selecting the appropriate CNN architecture depends on its intended application. ResNet50, in particular, stands out for its remarkable accuracy. This architecture introduces a novel approach compared to earlier CNNs by employing residual

blocks and skip connections, effectively addressing the vanishing gradient problem[4].

The role of the barista is crucial in discerning the maturity level of coffee beans, as each level offers distinct benefits and flavours. For instance, coffee beans at a medium maturity level are typically favoured for the single-origin method. Therefore, to help baristas and coffee enthusiasts distinguish between the maturity levels of coffee beans, developing an application for classifying the maturity level of roasted coffee beans is deemed necessary.

Convolutional Neural Networks (CNNs) have significant applications in agriculture, especially in detecting coffee bean maturity. Various studies have utilized CNNs for tasks like coffee leaf disease detection[5] and coffee bean quality classification based on roast level[6]. These applications involve training CNN models on datasets containing images of coffee leaves, beans, or cherries to accurately classify diseases, maturity stages, or roast levels. By leveraging transfer learning and model optimization techniques, CNNs can achieve high accuracy rates, such as 98.38% for green coffee bean quality detection [4] and 99.01% for classifying coffee type and maturity [7]. Using CNNs in agriculture enables farmers to automate and efficiently process their crops, improving crop management and quality.

A smartphone application to detect coffee bean maturity must be designed with several crucial aspects in mind. First, the user interface must be intuitive and easy to understand so that users can quickly access the coffee bean maturity detection. Furthermore, the application should allow users to take pictures of coffee beans with high quality and sufficient resolution. After capturing the image, the application must be able to perform image analysis using image processing and deep learning algorithms, such as Convolutional Neural Networks (CNN), which have been trained to recognize visual characteristics that indicate the ripeness of coffee beans. The results of this analysis then need to be conveyed to users clearly, either through visual displays or easy-to-understand text messages[8].

The research problem is how to implement the Convolutional Neural Networks (CNN) in an Android-based application to detect the maturity of coffee beans. Despite achieving high accuracy in controlled environments, deploying such models in real-world scenarios presents significant challenges. These include inconsistent image quality due to varying camera specifications and environmental conditions, limited diversity in training data leading to overfitting, and the computational constraints of mobile devices, which hinder the performance of complex CNN models in real-time applications. The dynamic lighting conditions, background noise, and ensuring a user-friendly interface for consistent data capture further complicate the problem.

This research aims to address these challenges and improve the real-time accuracy of the CNN model when deployed on mobile devices. It involves enhancing the robustness of the model through data augmentation and regularization techniques, leveraging transfer learning, and optimizing the model for mobile performance through compression and edge computing solutions. The research aims to develop adaptive algorithms for managing environmental variability, establish efficient preprocessing pipelines, and

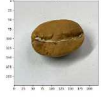
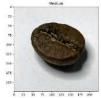
continuously integrate user feedback mechanisms to refine the model. Ultimately, the goal is to create a reliable, user-friendly application that accurately detects coffee bean maturity in diverse real-world conditions.

II. DESIGN AND IMPLEMENTATION

A. Design

Data collection was taken from the Kaggle Sakdipat Ontoum website[9]. Each of the 900 data sets is divided into three levels, as shown in TABLE I.

TABLE I. DATASET

Roasted Coffee Bean Maturity Level	Data Amount	Sample
Light	300	
Medium	300	
Dark	300	

The dataset used in this model will be divided into 80% for training data and 20% for testing data[10]. Fig.1. depicts the system design block diagram, encompassing the model creation and application development processes. The model creation process, executed in TensorFlow, commences with image acquisition, followed by model training utilizing ResNet50, a convolutional neural network architecture. Subsequently, the trained model is saved in TensorFlow Lite (.tflite) format. This .tflite model is then integrated into an Android application to classify the maturity level of roasted coffee beans captured via a cellphone camera. The model identifies the maturity level of the roasted coffee beans, providing classification results accordingly.

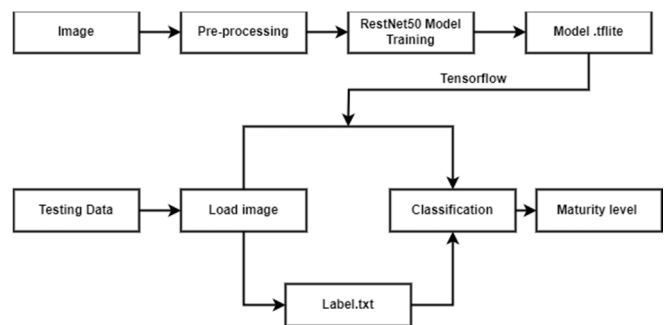


Fig.1. System design

The design of the roasted coffee level application comprises three stages: input, process, and output. The input stage involves utilizing images of roasted coffee beans at various maturity levels as datasets for training and evaluating the performance of the ResNet-50 CNN model. The stages of designing and constructing this application encompass two

main processes: creating the CNN ResNet-50 model and developing the application.

Training a ResNet-50 CNN model for detecting coffee bean maturity involves several key steps. First, a labelled dataset is prepared and augmented to increase diversity. The pre-trained ResNet-50 model, excluding the top layers, is loaded, and custom classification layers are added. The model is then compiled with an optimizer and loss function suitable for multi-class classification. Data generators are used for efficient data loading and augmentation during training. The model is trained using these generators and evaluated on a separate test set to assess performance—the fine-tuning performed by unfreezing some base model layers and retraining with a lower learning rate. Finally, the trained model is saved for deployment in the Android application. The creation process of the ResNet-50 CNN model initiates with preprocessing and culminates in generating a TFLite model. This TFLite model is then integrated into the application development phase, enabling the application to classify the maturity level of roasted coffee beans by leveraging the camera feature on an Android cellphone.

The preprocessing process commences with dataset preparation. Initially, the dataset is acquired from the Kaggle website and imported into the program. Subsequently, the dataset undergoes an image augmentation stage. Image augmentation enriches and diversifies the dataset by applying a range of transformations to the original images. This process enhances the variety and quality of the image dataset utilized for model training. Parameters utilized in the augmentation process include rotation, horizontal and vertical shifts, height shifts, cropping, and horizontal flipping.

The CNN model previously designed proceeds to the training process, which utilizes TensorFlow. This phase aims to assess the performance of the CNN model. Parameters such as learning rate and epochs are crucial in this training process. The learning rate value influences the updating of model weights during training, while each epoch represents one complete pass through the training dataset. Initially, the model undergoes training to determine its loss value and accuracy. Subsequently, the trained model is tested to evaluate its performance, generating validation loss and accuracy values. These values serve as metrics to assess the effectiveness of the model created during the training process.

CNN models developed using TensorFlow are converted into TensorFlow Lite (TFLite) models. This conversion process enables the model to be deployed on devices with limited resources, particularly mobile devices like Android smartphones. Converting the model to TFLite format reduces its size, optimizing its performance on mobile devices.

B. System Implementation

In the proposed ResNet-50 architecture shown in Fig.2, the system receives a dataset comprising 1000 inputs, each loaded sequentially. Subsequently, the prediction process is initiated using the trained ResNet-50 model. During this process, feature extraction is conducted with the assistance of convolutional and pooling layers. The extracted features undergo dimensionality reduction through a flattening function, enabling them to be processed in the fully connected layer.

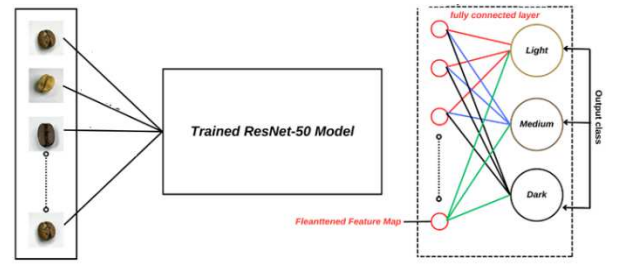


Fig.2. ResNet-50 architecture

For smartphone applications, users have two options to classify roasted coffee bean maturity levels. Firstly, they can access the gallery and select an image of roasted coffee beans for classification. Alternatively, users can utilize the cellphone camera to capture photos of the roasted coffee beans to be classified. The implementation is shown in Fig.3.

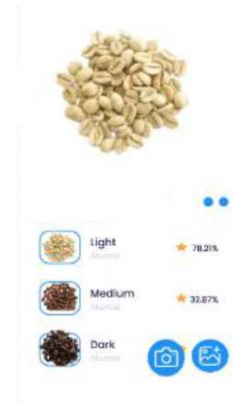


Fig.3 Smartphone application implementation

III. RESULTS AND DISCUSSION

Testing is conducted to evaluate the system's performance, ensuring it functions as intended and delivers accurate and consistent results. The testing in this research is divided into two parts: firstly, testing the pre-trained ResNet-50 model, and secondly, testing the application's ability to recognize the maturity level of roasted coffee beans.

The testing of the pre-trained ResNet-50 model aims to evaluate its performance. This testing process consisted of two stages: firstly, testing with variations in training parameters, and secondly, evaluation of the confusion matrix. During the first stage, various training parameters such as dataset, dropout layer, and number of epochs were adjusted to identify the optimal configuration. Once the best parameters were determined, the model underwent data testing using a confusion matrix. ResNet-50, a pre-trained model trained on the ImageNet dataset, is well-suited for transfer learning in image processing tasks.

The initial test conducted on the model involved variations in the dataset used. TABLE II illustrates four dataset comparisons, namely 60:40, 70:30, and 80:20. Each comparison indicates the percentage of the dataset allocated for training and testing, with 50 epochs provided for all tests.

In the first dataset variation, with a 60:40 ratio, 60% of the dataset served as training data, while the remaining 40% was used for testing. As depicted in TABLE II, this variation's accuracy and validation accuracy values were 99.44% and 96.67%, respectively, with corresponding loss and validation loss values of 1.31 and 9.28. Testing a dataset with a 70:30

ratio resulted in higher accuracy and validation accuracy than previous tests, with 98.89% and 97.78%, respectively. This dataset's loss and validation loss values were also 3.26 and 7.69. The final dataset tested, with an 80:20 ratio, exhibited the highest accuracy among the dataset variations, reaching 98.19%, with a validation accuracy of 97.22%. Moreover, this variation demonstrated the lowest loss and validation loss values, at 5.04 and 7.99, respectively.

TABLE II. MODEL TESTING BASED ON DATASET VARIATIONS

No	Data split	Learning rate	Accuracy	Accuracy validation	Loss	Loss validation
1	60:40	0.01	99.44	96.67	1.31	9.28
2	70:30	0.01	98.89	97.78	3.26	7.69
3	80:20	0.01	98.19	97.22	5.04	7.99

The results of model testing across various dataset variations indicate that utilizing a 60:40 ratio for training and validation yields the highest accuracy, best validation accuracy, and lowest loss values compared to other dataset variations. This ratio strikes an optimal balance between training the model to recognize data patterns and evaluating its performance.

The subsequent test compared the model's performance with and without the dropout layer during training. The results revealed that the model without the dropout layer exhibited higher accuracy and validation accuracy values than the model with the dropout layer, as seen in TABLE III. Specifically, the model without dropout achieved accuracy and validation accuracy values of 99.67% and 96.67%, with corresponding loss and validation loss values of 1.31 and 8.19. Conversely, the model utilizing the dropout layer achieved accuracy and validation accuracy values of 94.59% and 98.22%, with loss and validation loss values of 10.72 and 12.89.

These findings suggest that including dropout layers in the training process is less effective. It could be attributed to the nature of dropout, which can slow down the training process by deactivating certain units in each iteration. Consequently, the model employing dropout layers may require more epochs to attain the same level of accuracy as the model without dropout.

TABLE III. MODEL TESTING BASED ON THE USE OF DROPOUT LAYERS

Dropout	Learning Rate	Dataset %	Akurasi	Validasi Akurasi	Loss	Validasi Loss
Tanpa dropout	0.01	60:40	99.44	96.67	1.31	9.28
Dropout 0.5	0.01	60:40	94.59	95.22	10.72	12.89

The subsequent test involved evaluating the performance of the classification model using a confusion matrix. The model utilized was ResNet-50, with a learning rate of 0.01. Testing was conducted by dividing the dataset into a 60:40 ratio for training and validation data, and the training process was carried out for 50 epochs. This model selection was based on results from previous tests that showed superior training accuracy.

Fig. 5 shows a 3x3 confusion matrix table with two labels: actual and predicted values. The actual labels are divided into

three columns, and the prediction labels are represented in three columns. The confusion matrix results determine the accuracy, precision, recall, and F1-score evaluation metrics. These evaluation metric values are shown in TABLE IV and TABLE V.

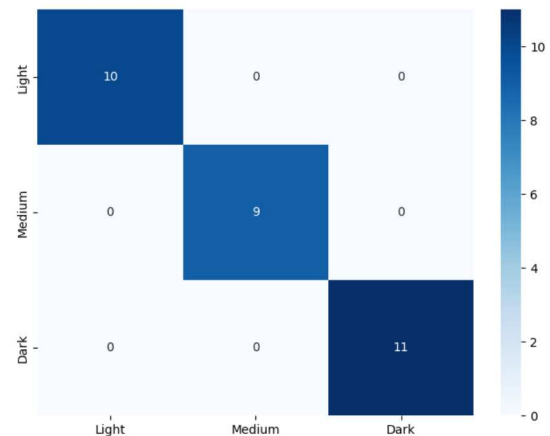


Fig.4 Confusion Matrix

TABLE IV. MODEL TESTING BASED ON THE USE OF DROPOUT LAYERS

Class	Precision	Recall	F1-score	Support
Light (0)	1.00	1.00	1.00	10
Medium (1)	0.99	0.99	0.99	9
Dark (2)	1.00	1.00	1.00	11

Two general methods are used to calculate the average evaluation metrics in multi-class classification: the macro average (macro Avg) method and the weighted average (weighted Avg) method. The macro average method calculates the evaluation metrics for each class separately and then averages these metrics. In contrast, the weighted average method also calculates the evaluation metrics for each class separately but then averages them, considering the proportion of each class in the dataset. The model evaluation results show an accuracy level of 100%, with precision, recall, and F1-score values all at 100%

TABLE V. ACCURACY, MACRO AVG DAN WEIGHTED AVG MODEL

Accuracy	Macro Avg	Weighted Avg	Support
1.00	1.00	1.00	30

After testing the model, the next step is to test the developed application. Application testing is a crucial stage in software development, ensuring the application operates correctly, meets specified functional and non-functional requirements, and provides an optimal user experience. This application testing was conducted on mobile devices running the Android operating system.

Testing the classification of the level of maturity of roasted coffee beans using the application is conducted by pointing the camera at the image object and using the gallery icon to view the classification results. The test results show the classification outcomes for each class in the application. Ten image samples were tested for the 'Light' class, and all results were correct. For the 'Medium' class, out of 10 image samples, 3 were incorrectly classified, and 7 were correctly classified. For the 'Dark' class, with 10 image samples, there were 2 incorrect and 8 correct classifications. There were 25 correct

and 5 incorrect classifications from 30 image samples. It indicates that the application achieved an accuracy rate of 83.33% in classification testing.

The accuracy drop from 100% to 83.33% when deploying your coffee classification model to an Android app can be attributed to several factors: the model might be overfitted to the training data, leading to poor generalization on new data; differences in lighting conditions between the training and real-world environments can significantly impact model performance; and variations in image quality, such as resolution and clarity, can also affect accuracy. Addressing these issues involves enhancing the robustness of the model through techniques like data augmentation and regularization, ensuring consistent preprocessing steps between training and deployment, and continuously evaluating and fine-tuning the model with real-world data from the app.

Fig.5 shows the confusion matrix from the classification test for the level of maturity of roasted coffee beans by application.

		Predicted Values		
		Light	Medium	Dark
Actual Values	Light	10	0	0
	Medium	0	7	3
	Dark	0	2	8

Fig.6. Confusion matrix for application testing

Implementing Convolutional Neural Networks (CNN) in an Android-based application for detecting coffee bean maturity faces several challenges. Key issues include inconsistent image quality due to varying camera specifications and environmental conditions, limited and less diverse training data, and the risk of model overfitting. Additionally, computational constraints of mobile devices, such as limited processing power and memory, hinder the deployment of complex CNN models for real-time applications. Environmental factors like dynamic lighting conditions and background noise further complicate accurate detection while ensuring a user-friendly interface and consistent data capture remain challenging.

Future directions to improve real-time accuracy focus on enhancing data collection and model robustness. Techniques such as data augmentation and crowdsourcing can increase the diversity of training datasets, while regularization and transfer learning can prevent overfitting. Model compression and edge computing solutions can optimize performance for mobile devices, and adaptive algorithms can help manage environmental variability. Additionally, implementing efficient preprocessing pipelines and integrating user feedback mechanisms can refine the model over time. Exploring IoT integration and ensuring cross-platform compatibility can also extend the application's usability and effectiveness in real-world scenarios.

IV. CONCLUSION

An Android-based application for classifying the maturity level of roasted coffee beans was successfully designed and built by integrating a Flutter-based Android application with a deep learning model. The integrated design and deep learning model can detect and classify the maturity level of roasted coffee beans in real time. The performance of the Android-based application was evaluated using a Convolutional Neural Network (CNN). The evaluation results show that this application can classify the maturity level of roasted coffee beans with an accuracy of 83%. Additionally, the application can classify the maturity level of roasted coffee beans in real time by integrating Flutter with the classification model. Consequently, this application provides a solution for classifying the maturity level of roasted coffee beans anytime and anywhere, and it performs this task in real time.

REFERENCES

- [1] W. Wang and Y. Yang, "Development of Convolutional Neural Network and its Application in Image Classification: a Survey," *Opt. Eng.*, vol. 58, no. 04, pp. 1–19, 2019, doi: 10.1117/1.oe.58.4.040901.
- [2] A. Faizah, P. H. Saputro, A. J. Firdaus, and R. N. R. Dzakiyullah, "Implementation of the Convolutional Neural Network Method to Detect the Use of Masks," *IJIS Int. J. Informatics Inf. Syst.*, vol. 4, no. 1, pp. 30–37, Mar. 2021, doi: 10.47738/ijis.v4i1.75.
- [3] M. F. Aslan, "Comparative Analysis of CNN Models and Bayesian Optimization-Based Machine Learning Algorithms in Leaf Type Classification," *Balk. J. Electr. Comput. Eng.*, vol. 11, no. 1, pp. 13–24, Jan. 2023, doi: 10.17694/BAJECE.1174242.
- [4] S. Raveena and R. Surendran, "ResNet50-based Classification of Coffee Cherry Maturity using Deep-CNN," *Proc. - 5th Int. Conf. Smart Syst. Inven. Technol. ICSSIT 2023*, pp. 1275–1281, 2023, doi: 10.1109/ICSSIT55814.2023.10061006.
- [5] K. Mridha *et al.*, "Explainable Deep Learning for Coffee Leaf Disease Classification in Smart Agriculture: A Visual Approach," *2nd IEEE Int. Conf. Distrib. Comput. Electr. Circuits Electron. ICDCECE*, 2023, 2023, doi: 10.1109/ICDCECE57866.2023.10151399.
- [6] G. Grimaldo, H. Rodriguez, and V. L. Cabrera, "Convolutional Neural Network Model for the Detection of Diseases and Pests in Coffee Crops," *Proc. - 2022 8th Int. Eng. Sci. Technol. Conf. IESTEC 2022*, pp. 684–690, 2022, doi: 10.1109/IESTEC54539.2022.00113.
- [7] G. Vilcamiza, N. Trelles, L. Vines, and J. Oliden, "A coffee bean classifier system by roast quality using convolutional neural networks and computer vision implemented in an NVIDIA Jetson Nano," *2022 Congr. Int. Innov. y Tendencias en Ing. CONIITI 2022 - Conf. Proc.*, 2022, doi: 10.1109/CONIITI57704.2022.9953636.
- [8] A. Sujiwa and I. Santoso, "Automatic Coffee Maker Machine Based on Internet of Things (IoT)," *BEST J. Appl. Electr. Sci. Technol.*, vol. 4, no. 01, p. 1, 2022, doi: 10.36456/best.vol4.no01.5343.
- [9] S. Ontoum, T. Khemanantakul, P. Sroison, T. Triyason, and B. Watanapa, "Coffee Roast Intelligence," 2022. doi: 10.48550/ARXIV.2206.01841.
- [10] V. Roshan, J. H. M. Stewart, R. Joseph, and H. M. Stewart, "Optimal Ratio for Data Splitting," *Stat. Anal. Data Min. ASA Data Sci. J.*, vol. 15, no. 4, pp. 531–538, Aug. 2022, doi: 10.1002/SAM.11583.